

JsonToGraph

Estructura del JSON de Salida

Este documento describe el formato JSON que produce **JsonToGraph** al convertir el JSON del sistema (variables + expressions + functions) en un **hipergrafo CSP** listo para propagación de dominio. Cada sección detalla la estructura de los nodos, sus campos, tipos de datos e invariantes que garantizan la consistencia del grafo resultante.

1. Estructura Raíz

El JSON de salida contiene cuatro arreglos principales que representan los distintos componentes del hipergrafo CSP:

```
{
  "variables": [ <VarNode>, ... ],
  "functions": [ <FuncNode>, ... ],
  "constraints": [ <ConstraintNode>, ... ],
  "adjacency": [ <AdjEntry>, ... ]
}
```

2. variables[] — Nodos de Variable

Cada elemento es el objeto original del input más un campo `id` añadido automáticamente. El `id` es el índice 0-based en el arreglo de entrada. Las variables de tipo `set` se registran como literales para que el parser no las confunda con identificadores de variable.

```
{
  "id": 0,
  "name": "X",
  "type": "integer",
  "domain": [1, 10],
  "value": null
}
```

Campo	Origen	Descripción
id	generado	Índice global de la variable (clave de referencia)
name	input	Nombre de la variable
type	input	"integer", "float", "boolean", "set", ...
domain	input	Dominio inicial (array, intervalo, etc.)

Campo	Origen	Descripción
value	input	Valor fijo si ya está asignado, null si libre

3. functions[] — Nodos de Función

Cada elemento es el objeto original del input más un campo `id` (índice 0-based). Las funciones actúan como relaciones entre variables de entrada y variables de salida dentro del grafo de restricciones.

```
{
  "id":      0,
  "name":    "distancia",
  "inputs":  ["X", "Y"],
  "outputs": ["D"]
}
```

Campo	Origen	Descripción
id	generado	Índice global de la función
name	input	Nombre de la función
inputs	input	Variables de entrada
outputs	input	Variables de salida

4. constraints[] — Hiperedges del Grafo

Cada constraint proviene de un string en `expressions[i].constraints[j]`. Se parsea con `PrattParser` y se serializa su AST en post-orden. Esto permite reconstruir el árbol de expresión de forma eficiente durante la propagación de dominio.

```
{
  "id":      0,
  "expr":    "X + Y >= 10",
  "var_refs": [0, 1],
  "func_refs": [],
  "root":    4,
  "nodes":   [ <ASTNode>, ... ]
}
```

Campo	Tipo	Descripción
id	int	Índice global de la constraint (0-based, secuencial)
expr	string	Expresión original tal como estaba en el input
var_refs	int[]	IDs de variables que aparecen en la expresión (únicos)
func_refs	int[]	IDs de funciones referenciadas (únicos)
root	int	id del nodo raíz del AST dentro de nodes

Campo	Tipo	Descripción
nodos	ASTNode[]	Todos los nodos del AST en post-orden
error	string	Solo presente si el parseo falló

4.1 nodes[] — Nodos del AST (post-orden)

Los nodos hijos siempre tienen id menor que su padre. El nodo con id == root es la raíz de la expresión. El orden post-orden facilita la evaluación bottom-up sin necesidad de recursión adicional.

Nodo Number

```
{ "id": 0, "type": "Number", "value": 10 }
```

Nodo Variable

```
{ "id": 1, "type": "Variable", "name": "X" }
```

Nodo Boolean

```
{ "id": 2, "type": "Boolean", "value": true }
```

Nodo Operador Binario

Aplica a: Add, Subtract, Multiply, Divide, Modulo, Power, And, Or, Equals, NotEquals, Less, Greater, LessEq, GreaterEq, In

```
{ "id": 3, "type": "Add", "left": 1, "right": 0 }
```

Campo	Descripción
left	id del subárbol izquierdo
right	id del subárbol derecho

Nodo Operador Unario

Aplica a: Negate, Not

```
{ "id": 4, "type": "Negate", "operand": 1 }
```

Nodo Interval

```
{ "id": 4, "type": "Interval", "lo": 0, "hi": 1, "lo_open": false, "hi_open": false }
```

Campo	Descripción
lo	id del nodo que representa el mínimo
hi	id del nodo que representa el máximo
lo_open	true si el extremo inferior es abierto (
hi_open	true si el extremo superior es abierto)

Nodo Set

```
{ "id": 5, "type": "Set", "elements": [0, 1, 2] }
```

Campo	Descripción
elements	Array de ids de los nodos elemento

Nodo FunctionCall

```
{ "id": 6, "type": "FunctionCall", "name": "distancia", "args": [1, 2] }
```

Campo	Descripción
name	Nombre de la función
args	Array de ids de los argumentos

5. adjacency[] — Índice Inverso Variable → Constraints

Permite saber, dado un var_id, en qué constraints participa esa variable. Este índice es utilizado por el propagador CSP para determinar qué constraints deben re-evaluarse cuando el dominio de una variable cambia, optimizando el proceso de propagación.

```
{
  "var_id":      0,
  "name":        "X",
  "constraint_ids": [0, 2, 5]
}
```

Campo	Descripción
var_id	ID de la variable (mismo que en variables[])
name	Nombre en mayúsculas
constraint_ids	IDs de todas las constraints que referencian a esa variable

6. Invariantes Importantes

Las siguientes invariantes garantizan la integridad y consistencia del grafo generado:

1	Los id de variables son estables: el mismo var_id aparece en variables[i].id, en constraints[j].var_refs, y en adjacency
2	Si una variable aparece en N constraints, NO se duplica en variables[]; su var_id simplemente aparece en los var_re
3	Los nodos del AST son locales a cada constraint; el id de un nodo AST no tiene relación con los id de variables o fun
4	adjacency tiene exactamente una entrada por variable registrada, en el mismo orden que fueron encontradas en el in

7. Ejemplo Mínimo Completo

A continuación se muestra un ejemplo de extremo a extremo con dos variables enteras y una única constraint de igualdad. Este ejemplo ilustra cómo se construye el AST y cómo se vinculan los distintos componentes del grafo.

Input:

```
{
  "variables": [
    { "name": "X", "type": "integer", "domain": [1, 5] },
    { "name": "Y", "type": "integer", "domain": [1, 5] }
  ],
  "expressions": [
    { "constraints": ["X + Y = 6"] }
  ],
  "functions": []
}
```

Output:

```
{
  "variables": [
    { "name": "X", "type": "integer", "domain": [1, 5], "id": 0 },
    { "name": "Y", "type": "integer", "domain": [1, 5], "id": 1 }
  ],
  "functions": [],
  "constraints": [
    {
      "id": 0,
      "expr": "X + Y = 6",
      "var_refs": [0, 1],
      "func_refs": [],
      "root": 4,
      "nodes": [
        { "id": 0, "type": "Variable", "name": "X" },
        { "id": 1, "type": "Variable", "name": "Y" },
        { "id": 2, "type": "Add", "left": 0, "right": 1 },
        { "id": 3, "type": "Number", "value": 6 },
        { "id": 4, "type": "Equals", "left": 2, "right": 3 }
      ]
    }
  ],
  "adjacency": [
    { "var_id": 0, "name": "X", "constraint_ids": [0] },
    { "var_id": 1, "name": "Y", "constraint_ids": [0] }
  ]
}
```